Phase 27: Project Defense & Timed Pitch Scripts

Document Code: DEF-DOC-001

Project: Duplicate Bin Ecosystem

Target Audience: Hiring Managers, Principal BAs, Cross-Functional Interview Panels

1. Timed Pitch Scripts (Elevator Pitches)

30-Second Elevator Pitch

"Duplicate Bin is a hybrid enterprise storage optimization platform that solves unstructured data clutter and escalating cloud storage costs. It couples a privacy-first, local multi-threaded desktop utility that scans 10,000 files in under 19 seconds with a central FastAPI SaaS backend for remote fleet orchestration and credit-based monetization. As the Senior Technical BA, I reverse-engineered the domain, authored our BRD and FRD across 30 enterprise features, defined our zero-loss quarantine archive architecture, and established end-to-end traceability across 24 core requirements."

1-Minute Executive Pitch

"Unmanaged file duplicates consume between 15% and 35% of consumed storage across creative studios and corporate teams, triggering expensive recurring cloud storage tier jumps. Traditional cleanup tools fail because manual review takes hours and users fear accidentally deleting master files.
Duplicate Bin solves this with a two-pass size pre-filter and cryptographic hashing engine (SHA-256), 8 automated selection heuristics, and a safe quarantine archive with instant one-click rollback (`Ctrl+Z`). We also bridged this desktop utility to a central SaaS cloud platform via a 6-digit device pairing protocol and WebSockets.
In my role as Technical BA, I owned the requirements lifecycle—translating complex algorithmic and cryptographic needs into Gherkin acceptance criteria, modeling our REST and WebSocket API contracts, managing stakeholder trade-offs, and verifying zero data loss during UAT."

3-Minute Comprehensive Project Pitch

"During my tenure on the Duplicate Bin project, our mission was to solve storage sprawl and rising cloud storage overhead for creative prosumers and SME businesses. In digital asset environments, redundant RAW media, video exports, and project staging builds quickly overwhelm local SSDs and cloud storage.

The core challenge was threefold: first, speed—scanning tens of thousands of files without exhausting system memory; second, psychological—users refuse to clean drives if they fear losing originals; third, monetization—prosumers hate paying monthly subscriptions for tools they only use periodically.

I addressed these challenges systematically:

1. Algorithmic Requirements: I specified a two-pass detection pipeline where Pass 1 eliminates unique files by size in $O(n)$ time, and Pass 2 streams byte chunks into SHA-256 hashes backed by an SQLite WAL persistent cache, achieving an 87% time reduction on 100,000 files (from 3m 42s down to 28.4s).

2. Safety & Heuristics: To eliminate decision fatigue and fear, I specified 8 automated selection rules (such as 'Keep Best Quality' for RAW photos) and a Safe Archive Quarantine system that preserves folder hierarchies and provides instant `Ctrl+Z` undo.

3. Hybrid SaaS Architecture: I defined the API specifications for our FastAPI backend, introducing a frictionless 6-digit device pairing handshake and a Pay-As-You-Go credit billing ledger (10 GB free allowance).

I maintained an end-to-end Requirements Traceability Matrix linking all 24 formal requirements through unit testing and UAT, resulting in 100% acceptance sign-off with zero critical defects."*

5-Minute Technical BA & Architectural Pitch

(Use this for deep-dive technical interview rounds where panel members include Technical Leads and Lead Architects.)

*"Duplicate Bin represents an evolution from a standalone Python desktop utility into a commercial, distributed hybrid SaaS platform.

When analyzing the business domain, I identified that existing market solutions fall into two flawed extremes: either pure cloud web apps—which are useless for terabyte-scale local media because uploading files over broadband takes days and violates data privacy—or legacy command-line utilities like `fdupes`, which lack visual verification and risk catastrophic data deletion.

As Senior Technical BA, I structured our delivery into three architectural layers:

1. The Client Presentation Layer: Built on PySide6 (Qt6), featuring non-blocking asynchronous worker threads (`scan_worker.py`), drag-and-drop folder ingestion, 13 keyboard shortcuts, and a memory-bounded

threads (`scan_worker.py`), drag-and-drop folder ingestion, 13 keyboard shortcuts, and a memory-bounded

LRU preview cache that limits active photo thumbnails to 200 pixmaps.

2. The Core Computational Engine: Implements a two-pass detection pattern. Pass 1 collects file metadata via `os.walk` and groups candidates by exact byte size. Only size collisions proceed to Pass 2, where streaming chunk buffers (4KB to 1MB adaptive) calculate SHA-256 digests. Transitive duplicate clusters ($A=B, B=C$) are resolved using a Union-Find disjoint-set data structure with path compression, avoiding group fragmentation bugs. Persistent caching via SQLite in WAL mode reduces rescan times from 18.7s to 4.2s on 10k files.

3. The SaaS Management & Relay Tier: Built with FastAPI, SQLAlchemy, and WebSockets. I defined the API contract for a 6-digit numeric device pairing protocol that expires after 300 seconds, exchanging a temporary handshake code for a 64-character persistent token. The cloud relay enables central IT managers to dispatch remote scans (`CMD_START_SCAN`) and stream real-time progress (`AGENT_PROGRESS`) over WebSockets.

From a governance perspective, I authored the BRD, FRD, PRD, and Data Dictionary, modeled the business rules (such as Master File Preservation and System Directory Protection), and facilitated UAT with zero Sev-1 defects. This project demonstrated how a Technical BA bridges high-level commercial strategy with low-level systems engineering."*

2. Core Project Defense Q&A

What was your personal contribution?

"*Contribution to confirm based on candidate's actual involvement.* In this project context: I led the functional and technical requirements definition, translated business needs into Gherkin user stories, modeled data schemas and API contracts for the FastAPI backend, facilitated Change Control Board reviews, and authored the UAT test plan and Requirements Traceability Matrix."*

How did you handle conflicting stakeholders?

"Product wanted to enforce a strict cloud paywall on all desktop scans, whereas our prosumer community demanded a completely free offline utility. I resolved this by facilitating a trade-off matrix: we kept basic local scanning 100% free and unlimited, but monetized multi-device fleet pairing, remote scheduling, and aggregate cloud analytics via Pay-As-You-Go credit packs and team subscriptions."

What went wrong during the project, and what did you learn?

"In our early prototype, our image preview widget attempted to load full-resolution 45 MB RAW photos directly into Qt memory buffers, which rapidly caused Out-Of-Memory crashes when users browsed large archives. I recognized this as a critical unstated Non-Functional Requirement. I worked with the UI lead to implement an LRU Cache capped at 200 pixmaps and mandated the use of downsampled Pillow thumbnail decoders. This taught me that BAs must never assume memory constraints are purely an engineering detail—they directly dictate user experience and interface stability."

What would you do differently if starting over today?

"I would advocate for establishing the modular package architecture (`duplicate_bin/core`, `ui`, `engine`) from Day 1 rather than allowing our initial monolithic script (`scripts/duplicate_bin.py`) to grow to 6,200 lines before refactoring. Defining strict module boundaries early significantly accelerates parallel sprint delivery across frontend and backend teams."